Pràctica 1.1: Paral·lelització k-means

Autors:

Satxa Fortuny Pimentel
Nawfal Aissaoui El Aissaoui

Computació Paral·lela i Massiva / Grau en Enginyeria Informàtica

Escola Tècnica Superior d'Enginyeria / Universitat Rovira i Virgili

Març 2026

Índex

1 Introducció i Objectius **2**

2 Anàlisi de Dependències de Dades **3**
2.1 Graf de Dependències de la funció kmean . . . 3
2.2 Llistat i tipus de dependències a kmean . . . 4
2.3 Graf de Dependències de la funció qs . . . 5
2.4 Llistat i tipus de dependències a qs . . . 6

3 Transformacions i Estratègia de Paral·lelització **8**
3.1 Privatització de variables escalars temporals . . . 8
3.2 Operacions de Reducció . . . 8
3.3 Transformació dels vectors fS i fA . . . 8
3.4 Estratègia de Paral·lelització de la funció qs . . . 10

4 Avaluació del Rendiment **12**
4.1 Avaluació a la màquina Orca . . . 12
4.2 Avaluació a la màquina Teen . . . 12

5 Conclusions **13**

1 Introducció i Objectius

En aquest document es detall el procés de paral·lelització d'un codi. Consta d'un anàlisi teòric, una transformació i una avaluació del rendiment. Per dur a terme aquesta paral·lelització en una arquitectura de memòria compartida, s'ha fet ús de l'estàndard i la interfície de programació d'aplicacions OpenMP.
El programa consta de dues càrregues de treball principals:

- **Algorisme K-means:** Encarregat de calcular les comunitats. El problema processa un volum de 600000 elements, que s'han de classificar iterativament en 200 comunitats.
- **Algorisme QuickSort:** Mètode d'ordenació recursiu basat en el paradigma de Divide and Conquer. Utilitzat en la fas final del programa per ordenar els centroides obtinguts per l'algorisme K-means.

El punt de partida per a l'avaluació d'aquesta pràctica són els temps d'execució de la versió seqüencial base compilada amb el flag d'optimització màxima -O3. Les mètriques inicials preses són:

- **Teen:** 8.8 segons.
- **Orca:** 17.3 segons.

Un cop definit el context, els objectius principals de la pràctica són:

- **Anàlisi teòric:** Identificar les dependències de dades introduïdes per la paral·lelització i la generació d'un graf de dependències per estudiar amb més detall aquestes dependències i el seu tractament.
- **Preservació de la correcció:** Garantir que la paral·lelització del codi no altera el resultats d'aquest i a la vegada evitar condicions de carrera.
- **Maximització del rendiment:** Aplicar les estratègies de paral·lelització òptimes per minimitzar l'overhead i superar els llindars d'acceleració exigits. Concretament, l'objectiu és aconseguir:
  - Una mitjana harmònica de l'speedup >= a la màquina Teen, forçant la creació de fils des de 2 fins a 32.
  - Una mitjana harmònica de l'speedup >= a la màquina Orca, forçant la creació de fils des de 2 fins a 128.

2 Anàlisi de Dependències de Dades

Per poder dissenyar una estratègia de paral·lelització eficient i lliure de condicions de carrera, és imprescindible realitzar una anàlisi prèvia de les dependències de dades del codi seqüencial.

2.1 Graf de Dependències de la funció kmean

La part computacionalment més gran la té la funció kmean. D'aquí en endavant pel que respecta la funció kmeans parlarem de seqüencies les quals estaran numerades de la següent manera:

Listing 1: Codi kmeans

```
S1:     for (i=0;i<fN; i++)
S2:     {
S3:             min = 0;
S4:             dif = abs(fV[i] -fR[0]);
S5:             for (j=1;j<fK;j++)
S6:                     if (abs(fV[i] -fR[j]) < dif)
S7:                     {
S8:                             min = j;
S9:                             dif = abs(fV[i] -fR[j]);
S10:            }
S11:            fD[i] = min;
S12:    }
S13:    for(i=0;i<fK;i++)
S14:            fS[i] = fA[i] = 0;
S15:    for(i=0;i<fN;i++)
S16:    {
S17:            fS[fD[i]] += fV[i];
S18:            fA[fD[i]] ++;
S19:    }
S20:    dif = 0;
S21:    for(i=0;i<fK;i++)
S22:    {
S23:            t = fR[i];
S24:            if (fA[i]) fR[i] = fS[i]/fA[i];
S25:                    dif += abs(t - fR[i]);
S26:    }
S27:    iter ++;
```

S'ha dut a terme un anàlisi del bucle principal del programa per identificar les interaccions entre les diferents sentències. A partir del graf obtingut, es classifiquen les dependències en flux (δ), de sortida (δ^o) i anti dependències ($\delta^/$).

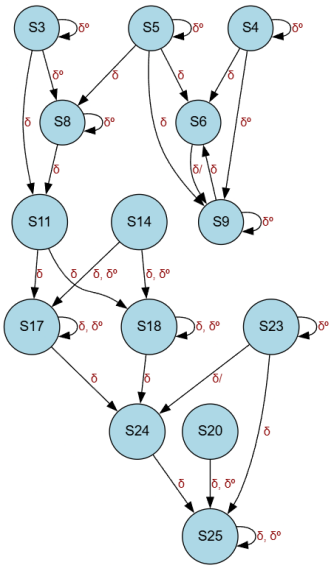

Figura 1: Graf de dependències de la funció kmean

2.2 Llistat i tipus de dependències a kmean

Com ja s'ha dit abans, s'han identificat 3 tipus de dependències de dades. Per facilitar-ne el tractament, s'han agrupat segons el seu tipus i tractament.

A) Dependències en variables escalars temporals i de control intern (min, t, j).

- **Dependències detectades:**
  - **min:** S3 (δ^o) S3, S3 (δ^o) S8, S3 (δ) S11.
  - **t:** S23 (δ^o) S23, S23 (δ) S25.
  - **j:** S5 (δ^o) S5, S5 (δ) S6, S5 (δ) S8, S5 (δ) S9.
- **Anàlisi:** Es produeixen dependències de sortida (δ^o) i de flux (δ) transportades per les iteracions dels bucles exteriors. Les variables min i t s'utilitzen com a emmagatzematge temporal d'un sol ús dins de cada iteració, mentre que j actua com a variable de control del bucle intern i es reinicialitza a cada pas del bucle exterior. Aquest patró indica que, en paral·lel, totes tres generarien condicions de carrera si es compartissin, de manera que requeriran privatització explícita (mitjançant la clàusula private per trencar les dependències i evitar que diferents fils les sobreescriguin simultàniament.

B) Dependències en la variable escalar d'estat (dif)

- **Dependències detectades:**
  - **Al primer bucle:** S4 (δ^o) S4, S4 (δ) S6, S4 (δ^o) S9, S6 ($\delta^/$) S9, S9 (δ) S6, S9 (δ^o) S9.
  - **Al quart bucle:** S20 (δ, δ^o) S25, S25 (δ, δ^o) S25.
- **Anàlisi:** La variable dif presenta una dualitat problemàtica. Al primer bucle actua com a variable temporal per calcular la distància. Al quart bucle, presenta dependències (δ, δ^o) consistents en una lectura i escriptura contínua (dif += ...). En ser una operació associativa i commutativa d'acumulació, s'identifica clarament com un patró de reducció.

C) Dependències en els vectors d'acumulació i assignació (fD, fS, fA)

- **Dependències detectades:**
  - **fD:** S11 (δ) S17, S11 (δ) S18.
  - **fS:** S14 (δ, δ^o) S17, S17 (δ, δ^o) S17, S17 (δ) S24.
  - **fA:** S14 (δ, δ^o) S18, S18 (δ, δ^o) S18, S18 (δ) S24.

- **Anàlisi:** L'escriptura a fD al primer bucle és segura (cada iteració escriu a un índex i diferent, sent un DOALL pur). Al tercer bucle, les matrius fS i fA s'accedeixen mitjançant un índex indirecte fD[i]. Atès que múltiples iteracions poden pertànyer al mateix clúster, apareixen dependències de sortida (δ^o) i de flux (δ) que impliquen que múltiples fils podrien intentar escriure simultàniament a la mateixa posició de memòria. Això serà el coll d'ampolla principal de l'aplicació. Apart, aquest tercer bucle s'ha de fer després de les inicialitzacions a 0, pel que s'haurà d'emprar un barrier.

D) Dependències en el vector de centroides (fR)

- **Dependències detectades:**
  - **fR:** S23 ($\delta^/$) S24, S24 (δ) S25.
- **Anàlisi:** La lectura i posterior escriptura de fR[i] al quart bucle genera una antidependència de distància 0, seguida d'una dependència de flux de distància 0 per avaluar la diferència a S25. Com que totes aquestes operacions es realitzen sobre el mateix índex i dins de la mateixa iteració del bucle, no generen dependències creuades entre fils. Així doncs, l'actualització de fR és un DOALL totalment paral·lelitzable de forma segura sense necessitat de sincronització addicional per índex.

E) Dependència del bucle exterior de Nivell 1 (do-while)

- **Anàlisi:** El bucle do-while depèn del valor final de la variable dif calculada a la iteració anterior. Això genera una dependència de flux (δ) transportada pel bucle que impedeix avaluar múltiples iteracions del do-while en paral·lel. Per tant, el primer nivell de l'algorisme serà estrictament seqüencial. En casos com aquest es podria considerar fer una paralel·lització exploratòria o especulativa pero donat que el número de camins és immens (al haver de fer tota la combinatòria de numero de punts per comunitat), no val la pena.

2.3 Graf de Dependències de la funció qs

La funció `qs` implementa una ordenació QuickSort in-place mitjançant un bucle `while` amb dues branques mútuament excloents (IF i ELSE) que avancen els índexs `i` i `f` cap al centre de la partició. D'aquí en endavant, les sentències del bucle s'enumeren de la següent manera:

Listing 2: Codi qs - bucle while

```
// BRANCA IF
S1:  fV[i-1] = vtmp;
S2:  fA[i-1] = vta;
S3:  i++;
S4:  vtmp = fV[i];
S5:  vta = fA[i];
// BRANCA ELSE
S6:  vfi = fV[f];
S7:  vfa = fA[f];
S8:  fV[f] = vtmp;
S9:  fA[f] = vta;
S10: f--;
S11: vtmp = vfi;
S12: vta = vfa;
```

Cal destacar que, com que les dues branques són mútuament excloents dins d'una mateixa iteració, totes les dependències entre sentències de branques diferents tenen distància mínima 1.

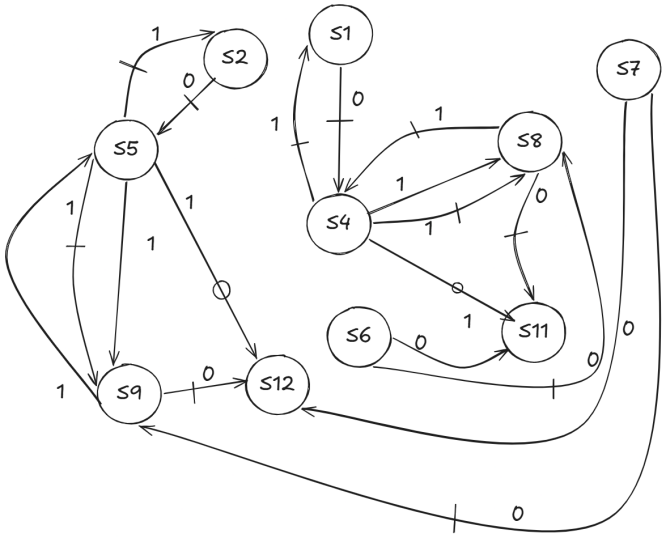

Figura 2: Graf de dependències de la funció qs

2.4 Llistat i tipus de dependències a qs

S'han identificat els mateixos tres tipus de dependències que a la funció `kmean`: de flux (δ), de sortida (δ^o) i anti-dependències ($\delta^/$).

A) Dependències en variables escalars temporals (`vtmp`, `vta`, `vfi`, `vfa`).

- **Dependències detectades:**
  - **vtmp:** S1 ($\delta^/\cdot 0$) S4, S4 ($\delta\cdot 1$) S8, S4 ($\delta^o\cdot 1$) S11, S8 ($\delta^/\cdot 0$) S11.
  - **vta:** S2 ($\delta^/\cdot 0$) S5, S5 ($\delta\cdot 1$) S9, S5 ($\delta^o\cdot 1$) S12, S9 ($\delta^/\cdot 0$) S12.
  - **vfi:** S6 ($\delta\cdot 0$) S11.
  - **vfa:** S7 ($\delta\cdot 0$) S12.
- **Anàlisi:** Les variables `vtmp` i `vta` actuen com a registres d'intercanvi entre les dues branques. Les anti-dependències de distància 0 (WAR) reflecteixen que el valor llegit al principi de la branca IF ha de ser vàlid fins que s'escrigui el nou dins la mateixa iteració. Les dependències de flux i de sortida de distància 1 entre branques IF i ELSE indiquen que el valor produït en una iteració és consumit o sobreescrit en la iteració següent, generant cicles en el graf. Les variables `vfi` i `vfa` presenten únicament dependències de flux de distància 0 dins de la branca ELSE, sense problemàtica afegida.

B) Dependències en els vectors `fV[]` i `fA[]`.

- **Dependències detectades:**
  - **fV[]:** S4 ($\delta^/\cdot 1$) S1, S6 ($\delta^/\cdot 0$) S8.
  - **fA[]:** S5 ($\delta^/\cdot 1$) S2, S7 ($\delta^/\cdot 0$) S9.
- **Col·lisions creuades quan `i` i `f` convergeixen:**
  - **fV[]:** S4 ($\delta^/\cdot 1$) S8 i S8 ($\delta\cdot 1$) S4 quan `i == f`.
  - **fA[]:** S5 ($\delta^/\cdot 1$) S9 i S9 ($\delta\cdot 1$) S5 quan `i == f`.
- **Anàlisi:** Les anti-dependències sobre `fV[]` i `fA[]` de distància 1 indiquen que cada iteració de la branca IF llegeix una posició que la iteració anterior ja ha escrit, imposant un ordre estricte d'execució. Les col·lisions creuades bidireccionals quan els índexs convergeixen (`i == f`) han estat verificades mitjançant una traça manual de l'espai d'iteracions. S'ha demostrat que una posició llegida per S4 serà sobreescrita per S8 a la iteració immediatament posterior (WAR), i viceversa, una posició escrita per S8 serà llegida per S4 en la següent iteració (RAW), creant un cicle tancat.

C) Justificació de la no-paral·lelització del bucle `while`.

- **Anàlisi:** La combinació de dependències de distància 1 entre les variables escalars i els vectors, juntament amb les col·lisions dinàmiques creuades bidireccionals entre branques, genera múltiples cicles en el graf de dependències. Tal com estableix la teoria, l'existència de cicles implica que el bucle **no és paral·lelitzable**. La paral·lelització de l'algorisme s'obté a través de les **crides recursives** `qs(ii, f, ...)` i `qs(i, fi, ...)` al final de la funció, les quals treballen sobre particions disjuntes de l'array i poden executar-se en paral·lel sense cap dependència entre elles.

3 Transformacions i Estratègia de Paral·lelització

Un cop identificades les dependències de dades al codi original, s'ha dissenyat una estratègia de paral·lelització basada en les directives d'OpenMP. L'objectiu d'aquesta fase és aplicar les transformacions necessàries per trencar les dependències conflictives, minimitzar l'overhead de creació de fils i evitar la contenció en l'accés a memòria, elements crítics per assolir la mitjana harmònica requerida.

3.1 Privatització de variables escalars temporals

Tal com es va identificar a l'anàlisi, les variables min i t i la variable de control j s'utilitzen com a memòria temporal exclusivament dins de les iteracions dels bucles exteriors. Si es mantinguessin com a compartides, generarien múltiples condicions de carrera (δ^o, $\delta^/$).
Per solucionar-ho, s'aplica una transformació d'entorn de dades mitjançant la clàusula private(min, t, j). D'aquesta manera, OpenMP crearà una instància independent d'aquestes variables a la memòria de pila de cada fil, trencant les dependències i permetent l'execució asíncrona de les iteracions.
Addicionalment, s'ha detectat que la variable dif s'utilitza amb dos propòsits diferents al codi original: com a registre de la distància al primer bucle, i com a acumulador d'error al quart. Per aïllar aquests comportaments i evitar interferències, s'aplica una transformació de còpia de variables, declarant una variable local temporal dins del primer bucle, que també serà privatitzada.

3.2 Operacions de Reducció

Al quart bucle de la funció kmean, la variable global dif s'actualitza contínuament amb la suma de les diferències dels centroides (dif += abs(...)). Aquesta dependència entre iteracions és un clar patró d'acumulació associativa i commutativa.
Per paral·lelitzar aquest bucle, s'elimina la dependència aplicant la clàusula reduction(+:dif). Això permet que cada fil operi sobre una còpia local i privada de l'acumulador i, en finalitzar el bucle, OpenMP s'encarregui de sumar de forma segura els resultats parcials de tots els fils en la variable global.

3.3 Transformació dels vectors fS i fA

El codi presenta dependències d'escriptura sobre posicions indexades indirectament (fS[fD[i]] += fV[i]).La solució trivial consistiria a protegir aquestes escriptures amb la directiva pragma omp atomic. No obstant això, aquesta aproximació destrossaria el rendiment i faria impossible assolir els objectius d'acceleració. Tenint $N = 600.000$ iteracions repartides entre fins a 128 fils, però només $G = 200$ posicions de memòria on escriure, l'ús d'operacions atòmiques provocaria:

- **Alta contenció:** Desenes de fils quedarien bloquejats constantment esperant el lock de la mateixa posició de l'array.
- **Fals compartiment:** Diferents posicions dels vectors fS i fA compartirien la mateixa línia de caché. Encara que dos fils intentessin escriure a índexs diferents, la coherència de caché obligaria a invalidar contínuament les línies, col·lapsant el bus de memòria.

Per evitar aquest coll d'ampolla, s'ha optat per una estratègia de còpies locals (implementada idealment a través de la reducció d'arrays d'OpenMP, com reduction(+:fS[:fK], fA[:fK])). Aquesta tècnica dota a cada fil de la seva pròpia còpia privada de 200 elements. Cada fil acumula les seves dades localment sense necessitat de cap sincronització ni panys. Un cop acabat el bucle, es realitza una reducció final per sumar les matrius locals a la matriu global, eliminant pràcticament tota la contenció i el fals compartiment.

Listing 3: Codi kmeans paral·lelitzat

```
void kmean(int fN, int fK, long fV[], long fR[], int fA[])
{
    int i, j, min, iter=0;
    long dif, t;
    long fS[G];
    int fD[N];

    do {
        dif = 0;

        #pragma omp parallel
        {
            #pragma omp for private(j, min)
            for (i=0; i<fN; i++) {
                min = 0;
                long local_dif = abs(fV[i] - fR[0]);

                for (j=1; j<fK; j++) {
                    if (abs(fV[i] - fR[j]) < local_dif) {
                        min = j;
                        local_dif = abs(fV[i] - fR[j]);
                    }
                }
                fD[i] = min;
            }
            #pragma omp for
            for(i=0; i<fK; i++) {
                fS[i] = 0;
                fA[i] = 0;
            }

            #pragma omp for reduction(+:fS[:fK], fA[:fK])
            for(i=0; i<fN; i++) {
                fS[fD[i]] += fV[i];
                fA[fD[i]] ++;
            }

            #pragma omp for private(t) reduction(+:dif)
            for(i=0; i<fK; i++) {
                t = fR[i];
                if (fA[i]) fR[i] = fS[i] / fA[i];
                dif += abs(t - fR[i]);
            }

        }
        iter++;
    } while(dif);

    printf("iter%d\n",iter);
}
```

3.4 Estratègia de Paral·lelització de la funció qs

Tal com s'ha demostrat a l'anàlisi de dependències, el bucle `while` de la funció `qs` no és paral·lelitzable a causa dels cicles presents al graf de dependències. No obstant això, l'algorisme QuickSort presenta un potencial de paral·lelisme natural: un cop realitzada la partició, les dues crides recursives `qs(ii, f, ...)` i `qs(i, fi, ...)` operen sobre subvectors completament disjunts i independents entre ells, sense cap dependència de dades. Per explotar aquest paral·lelisme s'utilitza el model de tasques (`task`) d'OpenMP, que permet encuar cada crida recursiva com una tasca lleugera dins del *pool* de fils existent. A diferència del constructor `sections`, que crea un nou equip de fils a cada invocació recursiva (provocant un *overhead* de *fork-join* repetit), les tasques reutilitzen els fils ja actius dins d'una única regió `parallel`. Això redueix dràsticament el cost de gestió de fils en algorismes recursius amb molts nivells de profunditat com el QuickSort.

La crida inicial a `qs` des del `main` s'envolta en una regió `parallel` amb `single`, de manera que el *pool* de fils es crea una única vegada. El fil que executa el `single` inicia la recursió, i les tasques generades a cada nivell es distribueixen automàticament entre tots els fils disponibles.

A més, s'aplica un doble control de granularitat per evitar la creació excessiva de tasques en particions petites:

- Un llindar exterior (`THRESHOLD`): si la partició original és menor que aquest valor, les crides recursives es fan de manera estrictament seqüencial, sense cap directiva OpenMP.
- Una clàusula `if` a cada `task`: encara que s'hagi superat el llindar exterior, cada subtasca individual només es genera com a tasca paral·lela si la seva subpartició és prou gran. Si no ho és, OpenMP l'executa *inline* dins del fil actual, evitant l'*overhead* de gestió de la cua de tasques.

Finalment, la directiva `taskwait` garanteix que ambdues subtasques finalitzin abans que la funció retorni, preservant la correcció de l'algorisme.

Listing 4: Codi qs paral·lelitzat amb tasques

```
#define THRESHOLD 50

void qs(int ii, int fi, long fV[], int fA[])
{
    int i, f;
    long pi, pa, vtmp, vta, vfi, vfa;

    pi = fV[ii];
    pa = fA[ii];
    i = ii + 1;
    f = fi;
    vtmp = fV[i];
    vta = fA[i];

    while (i <= f)
    {
        if (vtmp < pi) {
            fV[i-1] = vtmp;
            fA[i-1] = vta;
            i++;
            vtmp = fV[i];
            vta = fA[i];
        }
        else {
            vfi = fV[f];
            vfa = fA[f];
            fV[f] = vtmp;
            fA[f] = vta;
            f--;
            vtmp = vfi;
            vta = vfa;
        }
    }
    fV[i-1] = pi;
    fA[i-1] = pa;

    if (fi - ii > THRESHOLD) {
        #pragma omp task if(f - ii > THRESHOLD)
        { if (ii < f) qs(ii, f, fV, fA); }

        #pragma omp task if(fi - i > THRESHOLD)
        { if (i < fi) qs(i, fi, fV, fA); }

        #pragma omp taskwait
    }
    else {
        if (ii < f) qs(ii, f, fV, fA);
        if (i < fi) qs(i, fi, fV, fA);
    }
}
```

La invocació des del `main` es realitza de la manera següent, creant el *pool* de fils una única vegada:

Listing 5: Invocació de qs des del main

```
#pragma omp parallel
{
    #pragma omp single
    qs(0, G-1, R, A);
}
```

4 Avaluació del Rendiment

Per avaluar l'impacte de la paral·lelització s'han realitzat bateries de proves en dues arquitectures de memòria compartida diferents: la màquina **Teen** i la màquina **Orca**. Per garantir la fiabilitat de les mètriques, s'ha demanat accés exclusiu als nodes (evitant interferències d'altres usuaris) i s'han utilitzat els temps reals d'execució (*elapsed*).

L'acceleració o *Speedup* (S) s'ha calculat dividint el temps de la versió seqüencial original (T_s) entre el temps de la versió paral·lelitzada per a N fils (T_p). L'anàlisi de la topologia amb l'eina `lscpu` ha revelat que ambdues màquines operen sota un únic node NUMA lògic, evitant penalitzacions per distància de memòria.

4.1 Avaluació a la màquina Orca

La topologia de la màquina Orca està basada en un processador AMD Ryzen Threadripper PRO 3995WX, compost per **64 nuclis físics** equipats amb tecnologia SMT (*Simultaneous Multithreading*), la qual cosa ofereix un total de **128 fils lògics** de processament. El temps seqüencial base (T_s) ha estat de **17.28 segons**.

Nombre de fils	**Temps d'execució (s)**	**Speedup** ($S = 17.28/T_p$)
1 (Seqüencial Base)	17.28	1.00x
2	14.26	1.21x
4	7.15	2.42x
8	3.63	4.76x
16	1.87	9.24x
32	0.97	17.81x
64	0.52	33.23x
128	0.32	54.00x

Taula 1: Resultats d'escalabilitat - Màquina Orca (64 Cores / 128 Threads)

A la Taula 1 s'observa una escalabilitat excel·lent als primers nivells, reduint el temps gairebé a la meitat cada cop que es doblen els fils (des de 14.26s fins a 0.52s amb 64 fils). Això demostra l'eficiència de la clàusula `reduction` d'OpenMP, que ha evitat la contenció i el *false sharing*. L'Speedup màxim assolit amb 128 fils és de **54x**. Tenint en compte que l'acceleració màxima teòrica ve delimitada principalment pel nombre de nuclis físics (64), aconseguir un factor de 54x demostra una eficiència d'escalabilitat extraordinària, traient el màxim profit de l'SMT per amagar les latències de memòria de les iteracions.

4.2 Avaluació a la màquina Teen

La màquina Teen compta amb un processador AMD Ryzen 7 7700 de **8 nuclis físics** i **16 fils lògics** (tecnologia SMT). El temps seqüencial base (T_s) obtingut ha estat de **9.08 segons**. Per avaluar l'impacte de la sobrecàrrega, s'ha forçat una darrera execució amb 32 fils.

Nombre de fils	**Temps d'execució (s)**	**Speedup** ($S = 9.08/T_p$)
1 (Seqüencial Base)	9.08	1.00x
2	8.28	1.10x
4	4.14	2.19x
8	2.12	4.28x
16	1.10	8.25x
32 (Forçats)	**1.13**	**8.04x**

Taula 2: Resultats d'escalabilitat i Oversubscription - Màquina Teen (8 Cores / 16 Threads)

Les dades de la Taula 2 reflecteixen perfectament l'arquitectura del maquinari. L'escalabilitat purament física es manté fins als 8 fils. El salt de 8 a 16 fils (on el temps baixa de 2.12s a 1.10s) il·lustra l'aprofitament directe de l'SMT per part de l'algorisme K-means. No obstant això, en l'execució amb 32 fils, el temps d'execució pateix una regressió (pujant a 1.13s). Aquest fenomen és la demostració empírica de l'**Oversubscription** (sobresubscripció). Com que s'han llançat més fils de programari (32) que CPUs lògiques disponibles (16), el sistema operatiu es veu forçat a expropiar i intercanviar els fils als nuclis constantment (*context switching*), generant un *overhead* que penalitza el rendiment global.

5 Conclusions

En la aquesta pràctica s'ha dut a terme la paral·lelització d'una aplicació complexa formada pels algorismes K-means i QuickSort, seguint una metodologia rigorosa d'anàlisi, transformació i avaluació de rendiment mitjançant OpenMP.

L'anàlisi exhaustiu dels grafs de dependències ha resultat crític per garantir la correcció semàntica del codi original. Al `kmean`, el repte principal requeia en l'actualització de les estructures compartides. S'ha demostrat que la privatització de variables de control i, fonamentalment, l'ús de reduccions natives d'OpenMP sobre vectors ha estat la solució arquitectònica òptima. Aquesta decisió ha permès esquivar els bloqueigs de les directives `critical` i evitar la degradació per *false sharing* a la memòria cau, possibilitant que l'algorisme escalés fregant el límit teòric dels nuclis físics en ambdues arquitectures (assolint un *Speedup* de 54x sobre els 64 nuclis de la màquina Orca).

Pel que fa al `qs` (QuickSort), l'estudi de dependències va revelar cicles bidireccionals recurrents que feien inviable la paral·lelització del bucle iteratiu intern. La solució va recaure en l'explotació del paral·lelisme natural de l'arbre de recursió mitjançant el model de *tasques* (`task`) d'OpenMP. Amb l'aplicació intel·ligent de llindars de granularitat (*thresholds*), s'ha aconseguit frenar la creació excessiva de tasques en les subparticions petites, minimitzant un *overhead* de gestió que altrament hauria col·lapsat l'execució.

Finalment, l'avaluació empírica, combinada amb l'anàlisi topològic dels nodes, ha permès confirmar els fonaments de la computació paral·lela. S'han observat els efectes positius de l'arquitectura SMT (*Simultaneous Multithreading*) en tasques de còmput intensiu, així com els límits d'escalabilitat descrits per la Llei d'Amdahl i la pèrdua d'eficiència provocada per l'*oversubscription* en entorns amb limitació estricta de maquinari (experimentada a la prova de 32 fils a la màquina Teen).